SQL - Compléments

Opérations ensemblistes
Jointures externes

Opérations ensemblistes

Opérations ensemblistes : l'union

Cette opération permet de faire l'union de deux tables (généralement le résultat de deux clauses SELECT). L'union des tables T1 et T2 contient toutes les lignes appartenant à T1, ou à T2, ou au deux.

Opérations ensemblistes : l'union

Cette opération permet de faire l'union de deux tables (généralement le résultat de deux clauses SELECT). L'union des tables T1 et T2 contient toutes les lignes appartenant à T1, ou à T2, ou au deux.

T1 :

CLUB
Le Bouscat
BEC
St Bruno

T2 :

CLUB
Le Haillan
Le Bouscat
BEC
St Criq
Blanquefort

T1 union T2 :

CLUB
BEC
Blanquefort
Le Bouscat
Le Haillan
St Bruno
St Cricq

Opérations ensemblistes : l'union

Cette opération permet de faire l'union de deux tables (généralement le résultat de deux clauses SELECT). L'union des tables T1 et T2 contient toutes les lignes appartenant à T1, ou à T2, ou au deux.

T1 :

CLUB
Le Bouscat
BEC
St Bruno

T2 :

CLUB
Le Haillan
Le Bouscat
BEC
St Criq
Blanquefort

T1 union T2 :

CLUB
BEC
Blanquefort
Le Bouscat
Le Haillan
St Bruno
St Cricq

Important : T1 et T2 doivent avoir la même structure (nombre et type des colonnes).

Opérations ensemblistes : l'union

Cette opération permet de faire l'union de deux tables (généralement le résultat de deux clauses SELECT). L'union des tables T1 et T2 contient toutes les lignes appartenant à T1, ou à T2, ou au deux.

T1 :

CLUB
Le Bouscat
BEC
St Bruno

T2 :

CLUB
Le Haillan
Le Bouscat
BEC
St Criq
Blanquefort

T1 union T2 :

CLUB
BEC
Blanquefort
Le Bouscat
Le Haillan
St Bruno
St Cricq

Question. Pourquoi la table '**T1 union T2**' est-elle triée ?

Opérations ensemblistes : l'union

En SQL, l'opérateur d'union ensemblistes se note... **UNION** !

Opérations ensemblistes : l'union

En SQL, l'opérateur d'union ensemblistes se note... **UNION** !

Exemple. Liste des compositeurs et des interprètes (musiciens qui sont des compositeurs, des interprètes, ou les deux) :

```
SELECT Nom_Musicien, Prénom_Musicien
    FROM Musicien
        INNER JOIN Composer ON Musicien.Code_Musicien = Composer.Code_Musicien
UNION
SELECT Nom_Musicien, Prénom_Musicien
    FROM Musicien
        INNER JOIN Interpréter ON Musicien.Code_Musicien = Interpréter.Code_Musicien
```

Opérations ensemblistes : l'union

En SQL, l'opérateur d'union ensemblistes se note... **UNION** !

Exemple. Liste des compositeurs et des interprètes (musiciens qui sont des compositeurs, des interprètes, ou les deux) :

```
SELECT Nom_Musicien, Prénom_Musicien
    FROM Musicien
        INNER JOIN Composer ON Musicien.Code_Musicien = Composer.Code_Musicien
UNION
SELECT Nom_Musicien, Prénom_Musicien
    FROM Musicien
        INNER JOIN Interpréter ON Musicien.Code_Musicien = Interpréter.Code_Musicien
```

Question. Que se passe-t-il si l'on inverse nom et prénom dans le 2e SELECT ?...

Opérations ensemblistes : l'intersection

Cette opération permet de faire l'intersection de deux tables. L'intersection des tables T1 et T2 contient toutes les lignes appartenant à la fois à T1 et T2.

Opérations ensemblistes : l'intersection

Cette opération permet de faire l'intersection de deux tables. L'intersection des tables T1 et T2 contient toutes les lignes appartenant à la fois à T1 et T2.

T1 :

CLUB
Le Bouscat
BEC
St Bruno

T2 :

CLUB
Le Haillan
Le Bouscat
BEC
St Criq
Blanquefort

T1 intersection T2 :

CLUB
BEC
Le Bouscat

Opérations ensemblistes : l'intersection

Cette opération permet de faire l'intersection de deux tables. L'intersection des tables T1 et T2 contient toutes les lignes appartenant à la fois à T1 et T2.

T1 :

CLUB
Le Bouscat
BEC
St Bruno

T2 :

CLUB
Le Haillan
Le Bouscat
BEC
St Criq
Blanquefort

T1 intersection T2 :

CLUB
BEC
Le Bouscat

À nouveau : T1 et T2 doivent avoir la même structure, et le résultat sera trié.

Opérations ensemblistes : l'intersection

En SQL, l'opérateur d'intersection ensembliste se note **INTERSECT**.

Exemple. Liste des compositeurs qui sont également chefs d'orchestre :

```
SELECT Nom_Musicien, Prénom_Musicien
    FROM Musicien
        INNER JOIN Composer ON Musicien.Code_Musicien = Composer.Code_Musicien
INTERSECT
SELECT Nom_Musicien, Prénom_Musicien
    FROM Musicien
        INNER JOIN Direction ON Musicien.Code_Musicien = Direction.Code_Musicien
```

Opérations ensemblistes : la soustraction

Cette opération permet de faire la différence ensembliste de deux tables. La différence **T1 – T2** contient toutes les lignes de T1 qui n'appartiennent pas à T2.

T1 :

CLUB
Le Bouscat
BEC
St Bruno

T2 :

CLUB
Le Haillan
Le Bouscat
BEC
St Criq
Blanquefort

T1 – T2 :

CLUB
St Bruno

Là encore : T1 et T2 doivent avoir la même structure, et le résultat sera trié.

Opérations ensemblistes : la soustraction

En SQL, l'opérateur de différence ensembliste se note **EXCEPT**.

Exemple. Liste des compositeurs qui sont pas chefs d'orchestre :

```
SELECT Nom_Musicien, Prénom_Musicien
    FROM Musicien
        INNER JOIN Composer ON Musicien.Code_Musicien = Composer.Code_Musicien
EXCEPT
SELECT Nom_Musicien, Prénom_Musicien
    FROM Musicien
        INNER JOIN Direction ON Musicien.Code_Musicien = Direction.Code_Musicien
```

Jointures externes

Petit retour sur les jointures internes

Considérons le schéma relationnel suivant :

Etudiants (INE, nom, prénom, numVilleNaiss)
Villes (numVille, nomVille, codePostal)
Etudiants.numVilleNaiss référence Villes.numVille

Qu'affiche la requête suivante ?

```
SELECT nomVille, nom, prénom
FROM Villes
    INNER JOIN Etudiants ON Villes.numVille = Etudiants.numVilleNaiss
```

Petit retour sur les jointures internes

Considérons le schéma relationnel suivant :

Etudiants (INE, nom, prénom, numVilleNaiss)
Villes (numVille, nomVille, codePostal)
Etudiants.numVilleNaiss référence Villes.numVille
Tous les attributs sont obligatoires sauf numVilleNaiss

Qu'affiche la requête suivante ?

```
SELECT nomVille, nom, prénom
FROM Villes
    INNER JOIN Etudiants ON Villes.numVille = Etudiants.numVilleNaiss
```

Comment faire pour voir également les villes où aucun étudiant n'est né ? ou les étudiants dont on ne connaît pas la ville de naissance ?

Jointures externes

- Une jointure interne entre deux tables T1 et T2 permet de créer un ensemble de couples : chaque couple est composé d'un élément de T1 et d'un élément de T2 qui satisfont une certaine condition.

Jointures externes

- Une jointure interne entre deux tables T1 et T2 permet de créer un ensemble de couples : chaque couple est composé d'un élément de T1 et d'un élément de T2 qui satisfont une certaine condition.

- Une jointure externe entre deux tables T1 et T2 va permettre de créer un ensemble de couples ou singletons :
  - les couples sont identiques aux couples produits par une jointure interne (composés d'un élément de T1 et d'un élément de T2 qui satisfont une certaine condition)
  - les singletons peuvent être des éléments de T1 n'ayant aucun élément associé dans T2, ou des éléments de T2 n'ayant aucun élément associé dans T1, ou encore les deux (selon le format de jointure externe utilisé).

Jointures externes simples (deux tables)

Reprenons notre exemple :

Etudiants (INE, nom, prénom, numVilleNaiss)

Villes (numVille, nomVille, codePostal)

Etudiants.numVilleNaiss référence Villes.numVille

Tous les attributs sont obligatoires sauf numVilleNaiss

Jointures externes simples (deux tables)

Reprenons notre exemple :

Etudiants (INE, nom, prénom, numVilleNaiss)
Villes (numVille, nomVille, codePostal)
Etudiants.numVilleNaiss référence Villes.numVille
Tous les attributs sont obligatoires sauf numVilleNaiss

Même requête, mais en affichant également les villes « sans étudiants » :

```
SELECT nomVille, nom, prénom
FROM Villes
    LEFT OUTER JOIN Etudiants
        ON Villes.numVille = Etudiants.numVilleNaiss
```

Jointures externes simples (deux tables)

Reprenons notre exemple :

Etudiants (INE, nom, prénom, numVilleNaiss)
Villes (numVille, nomVille, codePostal)
Etudiants.numVilleNaiss référence Villes.numVille
Tous les attributs sont obligatoires sauf numVilleNaiss

Même requête, mais en affichant également les villes « sans étudiants » :

```
SELECT nomVille, nom, prénom
FROM Villes
    LEFT OUTER JOIN Etudiants
        ON Villes.numVille = Etudiants.numVilleNaiss
```

Left *: on garde toutes les lignes de la table située avant la jointure, même celles qui n'ont pas d'élément associé (les colonnes nom et prénom vaudront Null)*

Jointures externes simples (deux tables)

Reprenons notre exemple :

Etudiants (INE, nom, prénom, numVilleNaiss)
Villes (numVille, nomVille, codePostal)
Etudiants.numVilleNaiss référence Villes.numVille
Tous les attributs sont obligatoires sauf numVilleNaiss

Cette fois, on affiche également les étudiants dont la ville est inconnue :

```
SELECT nomVille, nom, prénom
FROM Villes
    RIGHT OUTER JOIN Etudiants
        ON Villes.numVille = Etudiants.numVilleNaiss
```

Jointures externes simples (deux tables)

Reprenons notre exemple :

Etudiants (INE, nom, prénom, numVilleNaiss)
Villes (numVille, nomVille, codePostal)
Etudiants.numVilleNaiss référence Villes.numVille
Tous les attributs sont obligatoires sauf numVilleNaiss

Cette fois, on affiche également les étudiants dont la ville est inconnue :

```
SELECT nomVille, nom, prénom
FROM Villes
    RIGHT OUTER JOIN Etudiants
        ON Villes.numVille = Etudiants.numVilleNaiss
```

Right *: on garde toutes les lignes de la table située après la jointure, même celles qui n'ont pas d'élément associé (la colonne nomVille vaudra Null)*

Jointures externes simples (deux tables)

Reprenons notre exemple :

Etudiants (INE, nom, prénom, numVilleNaiss)
Villes (numVille, nomVille, codePostal)
Etudiants.numVilleNaiss référence Villes.numVille
Tous les attributs sont obligatoires sauf numVilleNaiss

Cette fois, on affiche également les étudiants dont la ville est inconnue :

```
SELECT nomVille, nom, prénom
FROM Villes
    RIGHT OUTER JOIN Etudiants
        ON Villes.numVille = Etudiants.numVilleNaiss
```

Remarque. *On aurait pu naturellement écrire, de façon équivalente :*

[...] Etudiants LEFT OUTER JOIN Villes [...]

Jointures externes simples (deux tables)

Reprenons notre exemple :

Etudiants (INE, nom, prénom, numVilleNaiss)
Villes (numVille, nomVille, codePostal)
Etudiants.numVilleNaiss référence Villes.numVille
Tous les attributs sont obligatoires sauf numVilleNaiss

Et enfin, en « gardant tout le monde » :

```
SELECT nomVille, nom, prénom
FROM Villes
        FULL OUTER JOIN Etudiants
                ON Villes.numVille = Etudiants.numVilleNaiss
```

Full *: on garde toutes les lignes des tables situées avant et après la jointure (les colonnes absentes vaudront Null)*

La fonction « COALESCE »

Cette fonction permet d'afficher une valeur « autre que Null », notamment dans le cas des jointures externes :

```
SELECT COALESCE(nomVille, 'ville de naissance inconnue'), nom, prénom
FROM Villes
        RIGHT OUTER JOIN Etudiants
                ON Villes.numVille = Etudiants.numVilleNaiss
```

Pour chaque étudiant dont la ville de naissance est inconnue, la colonne nomVille affichera ville de naissance inconnue.

La fonction « COALESCE »

Cette fonction permet d'afficher une valeur « autre que Null », notamment dans le cas des jointures externes :

```
SELECT COALESCE(nomVille, 'ville de naissance inconnue'), nom, prénom
FROM Villes
        RIGHT OUTER JOIN Etudiants
                ON Villes.numVille = Etudiants.numVilleNaiss
```

Pour chaque étudiant dont la ville de naissance est inconnue, la colonne nomVille affichera ville de naissance inconnue.

Il est naturellement possible d'utiliser cette fonction dans d'autres contextes :

```
SELECT nom_Musicien, COALESCE(prénom_Musicien, 'prénom inconnu')
FROM Musicien
```

Jointures externes et regroupement

On voudrait connaître le nombre d'étudiants nés dans chaque ville, en conservant les villes dans lesquelles aucun étudiant n'est né...

Jointures externes et regroupement

On voudrait connaître le nombre d'étudiants nés dans chaque ville, en conservant les villes dans lesquelles aucun étudiant n'est né...

On pourrait alors écrire :

```
SELECT nomVille, COUNT(*) AS 'nombre d''étudiants'
FROM Villes
        LEFT OUTER JOIN Etudiants
                ON Etudiants.numVilleNais = Villes.numVille
GROUP BY nomVille, Villes.numVille
```

Jointures externes et regroupement

On voudrait connaître le nombre d'étudiants nés dans chaque ville, en conservant les villes dans lesquelles aucun étudiant n'est né...

On pourrait alors écrire :

```
SELECT nomVille, COUNT(*) AS 'nombre d''étudiants'
FROM Villes
        LEFT OUTER JOIN Etudiants
                ON Etudiants.numVilleNais = Villes.numVille
GROUP BY nomVille, Villes.numVille
```

Malheureusement, cette requête ne renvoie pas le résultat espéré... Voyez-vous pourquoi ?

Jointures externes et regroupement

On voudrait connaître le nombre d'étudiants nés dans chaque ville, en conservant les villes dans lesquelles aucun étudiant n'est né...

En effet, la solution correcte est :

```
SELECT nomVille, COUNT(Etudiants.INE) AS 'nombre d''étudiants'
FROM Villes
        LEFT OUTER JOIN Etudiants
                ON Etudiants.numVilleNais = Villes.numVille
GROUP BY nomVille, Villes.numVille
```

Cette fois, le COUNT du groupe correspondant à une ville sans étudiants va renvoyer 0, car la colonne INE d'un tel groupe vaut NULL !

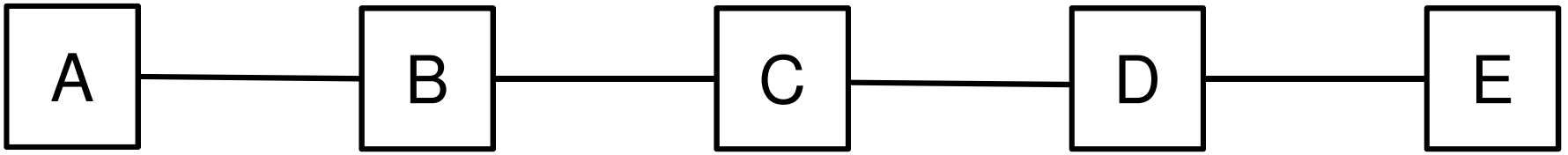
Méfiez-vous, ceci est une erreur classique ;-)...

Jointures externes avec au moins trois tables

Dès que le « chemin de jointure » est un peu long (donc au moins trois tables) il faut faire très attention à placer la jointure externe au bon endroit...

Le principe général est le suivant :

1. Considérer le chemin de jointure

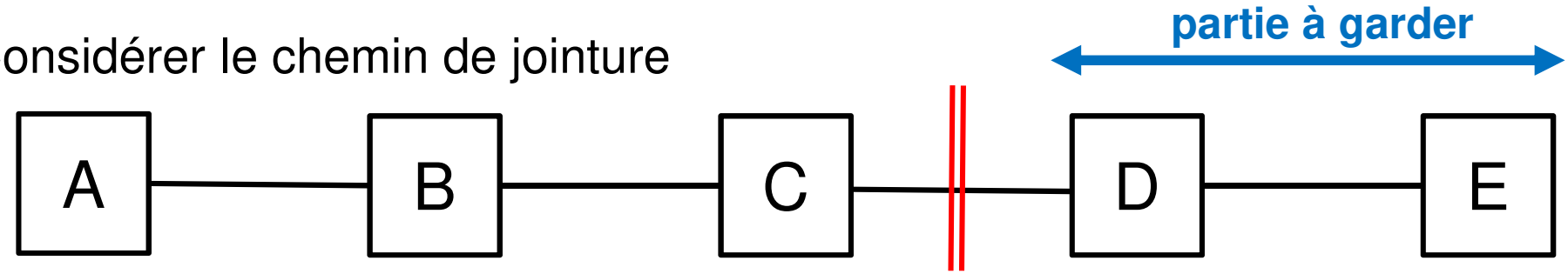


Jointures externes avec au moins trois tables

Dès que le « chemin de jointure » est un peu long (donc au moins trois tables) il faut faire très attention à placer la jointure externe au bon endroit...

Le principe général est le suivant :

1. Considérer le chemin de jointure



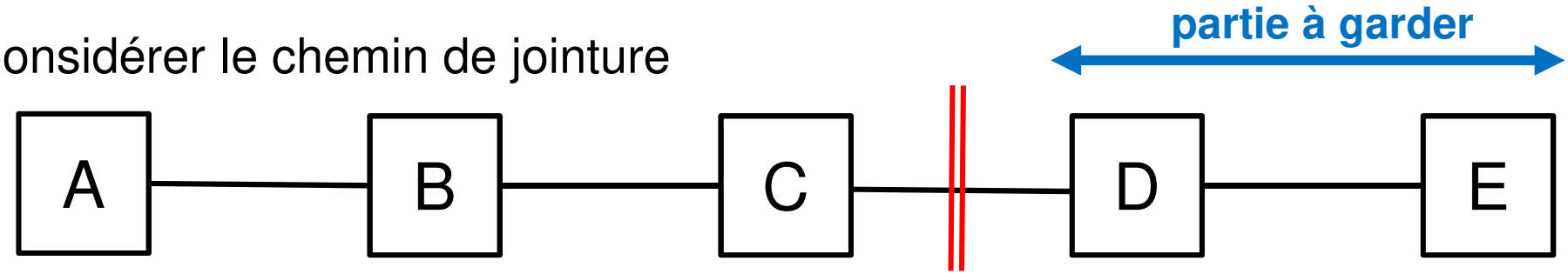
2. Repérer **le lien** au niveau duquel on souhaite tout garder, et **le côté à garder** (ici, on veut garder les (D,E), même si aucun C n'est associé au D...)

Jointures externes avec au moins trois tables

Dès que le « chemin de jointure » est un peu long (donc au moins trois tables) il faut faire très attention à placer la jointure externe au bon endroit...

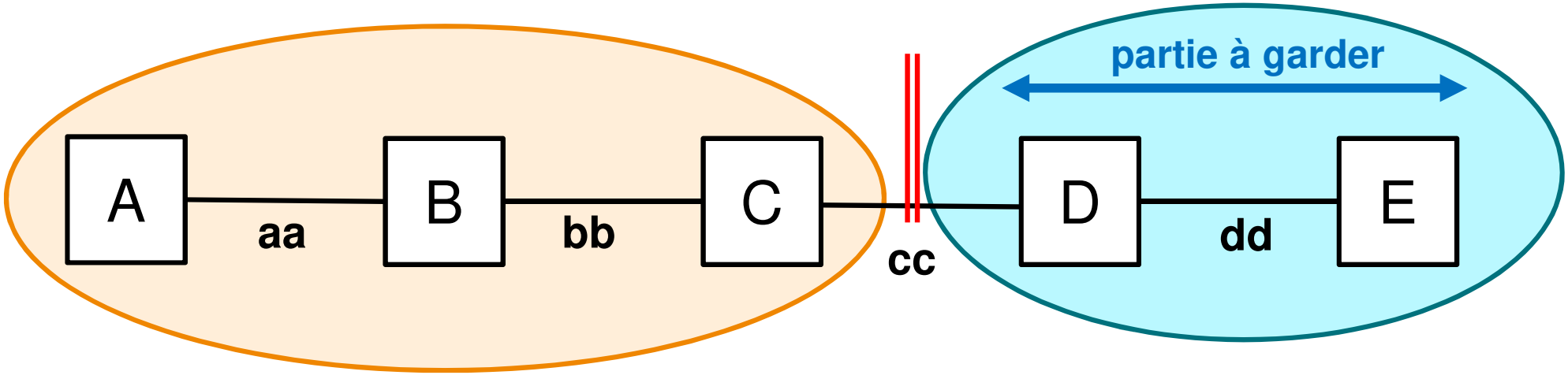
Le principe général est le suivant :

1. Considérer le chemin de jointure



2. Repérer **le lien** au niveau duquel on souhaite tout garder, et **le côté à garder** (ici, on veut garder les (D,E), même si aucun C n'est associé au D...)
3. Faire ensuite une jointure externe des deux parties (gauche et droite), en utilisant des parenthèses (chaque partie ne comprend que des jointures internes)

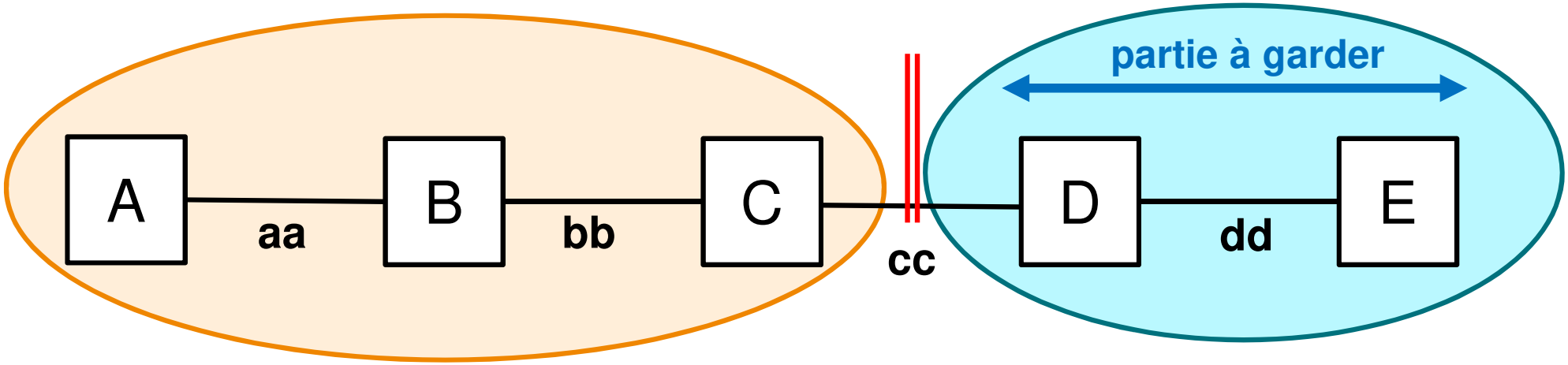
Jointures externes avec au moins trois tables



On obtient ainsi :

```
SELECT .....
FROM  ( A INNER JOIN B ON A.aa = B.aa
        INNER JOIN C ON B.bb = C.bb )
    RIGHT OUTER JOIN
        ( D INNER JOIN E ON D.dd = E.dd )
    ON C.cc = D.cc
```

Jointures externes avec au moins trois tables



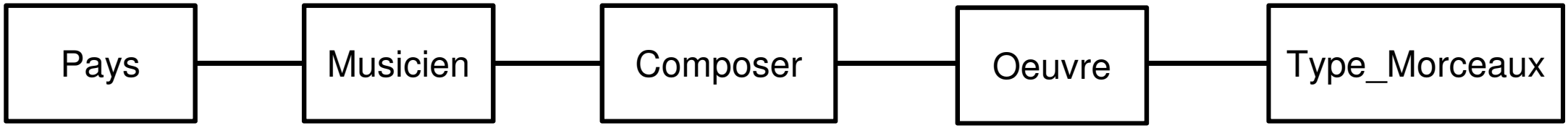
On obtient ainsi :

```
SELECT .....
FROM  ( A INNER JOIN B ON A.aa = B.aa
        INNER JOIN C ON B.bb = C.bb )
    RIGHT OUTER JOIN
        ( D INNER JOIN E ON D.dd = E.dd )
    ON C.cc = D.cc
```

Remarque. Si l'une des parties (**orange** ou **bleue**) ne contient qu'une seule table, les parenthèses pour cette partie sont facultatives...

Un exemple de jointure... complexe !

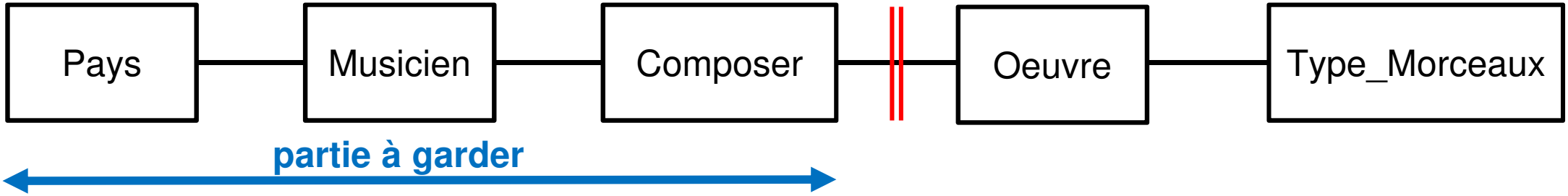
On veut afficher le nombre de sonates composées par des compositeurs italiens, en gardant les compositeurs italiens n'ayant pas composé de sonate...



Question. Sur quel lien doit s'effectuer la jointure externe ?...

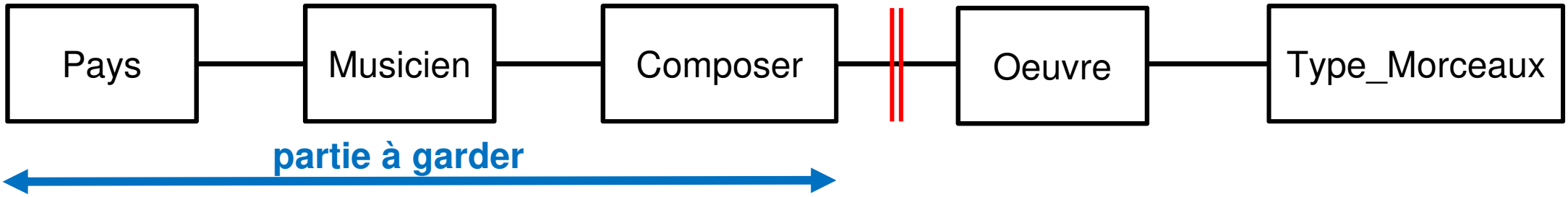
Un exemple de jointure... complexe !

On veut afficher le nombre de sonates composées par des compositeurs italiens, en gardant les compositeurs italiens n'ayant pas composé de sonate...



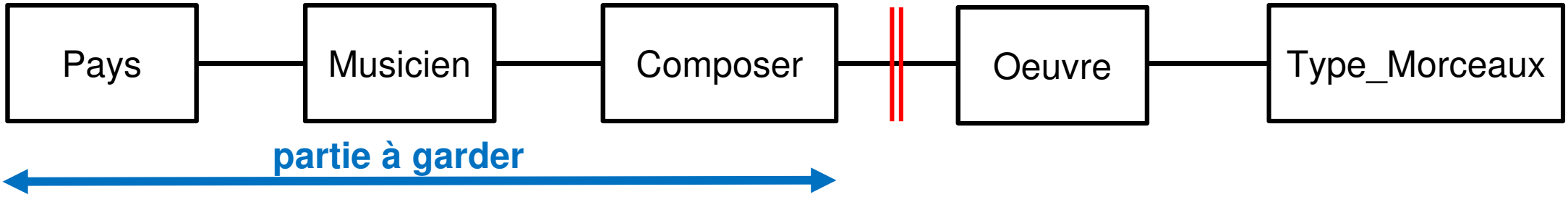
Question. Sur quel lien doit s'effectuer la jointure externe ? et quelle « partie » doit-on garder ?...

Un exemple de jointure... complexe !




```
SELECT Nom_Musicien, Prénom_Musicien, COUNT(O.Code_Oeuvre) AS NbSonates
FROM ( Pays AS P INNER JOIN Musicien AS M ON P.Code_Pays = M.Code_Pays
                                          AND Nom_Pays = 'Italie'
       INNER JOIN Composer AS C ON M.Code_Musicien = C.Code_Musicien )
     LEFT OUTER JOIN
       ( Oeuvre AS O INNER JOIN Type_Morceaux AS T ON O.Code_Type = T.Code_Type
                                                  AND Libellé_Type = 'Sonate' )
     ON C.Code_Oeuvre = O.Code_Oeuvre
GROUP BY M.Code_Musicien, Nom_Musicien, Prénom_Musicien
```

Un exemple de jointure... complexe !




```
SELECT Nom_Musicien, Prénom_Musicien, COUNT(O.Code_Oeuvre) AS NbSonates
FROM ( Pays AS P INNER JOIN Musicien AS M ON P.Code_Pays = M.Code_Pays
                                          AND Nom_Pays = 'Italie'
       INNER JOIN Composer AS C ON M.Code_Musicien = C.Code_Musicien )
     LEFT OUTER JOIN
       ( Oeuvre AS O INNER JOIN Type_Morceaux AS T ON O.Code_Type = T.Code_Type
                                                  AND Libellé_Type = 'Sonate' )
     ON C.Code_Oeuvre = O.Code_Oeuvre
GROUP BY M.Code_Musicien, Nom_Musicien, Prénom_Musicien
```

Thêta-jointure